

Practica 1 – File Manager

Víctor Manuel Peiró Martínez

FIIS 3A

Programación de Sistemas Distribuidos

Contenido

Introducción.....	3
Implementación Básica.....	3
Servidor	3
Cliente.....	4
Implementación Con Broker De Objetos	5
Configuración de AWS	5
Configuración Security Group	5
Broker	6
Conclusión	6

Introducción

La practica consiste en hacer un sistema de archivos distribuidos donde dos maquinas van a poder intercambiar y consultar archivos entre ambas maquinas.

Para ello, hemos implementado 2 soluciones como pedía el enunciado. La primera es una implementación más básica donde dos maquinas se conectan entre sí. La segunda implementa un bróker para que el cliente pueda elegir a que servidor quiere conectarse.

Implementación Básica

Esta implementación cuenta con solo 2 terminales conectados entre si para formar un claro ejemplo de un sistema cliente-servidor. Para no tener que estar trabajando con IPs, hemos decidido que ambos terminales deben estar en el mismo ordenador y se interconectaran mediante la IP de localhost siendo esta 127.0.0.1.

A continuación, se explica las funcionalidades de tanto cliente como servidor.

Servidor

Para el servidor se utilizan los siguientes archivos:

- **Ficheros utils:** Fichero con funciones de utilidad que nos ayudan sobre todo con asuntos relacionados con la emisión y recepción de datos entre terminales.
- **server.h:** Archivo que contiene la plantilla de la clase servidor además del enumerado fileFuncs y alguna función auxiliar de utilidad. Presente en ambos ejecutables ya que el cliente necesita acceso a las funciones auxiliares y al enumerado
 - **Enumerado fileFuncs:** Contiene todos los tipos de mensajes que se pueden intercambiar entre cliente-servidor. Cada mensaje invoca una funcionalidad del servidor que el cliente enviara para ejecutar en el servidor.
- **server.cpp:** Implementación del servidor y de cómo atiende las peticiones del cliente. Contiene el main del servidor.
 - **Atributos clients y connectionIds:** Contiene todos los clientes conectados junto a sus ids
 - **Función atenderCliente:** Atiende las peticiones del cliente en función hasta que se cierre la conexión.
- **filemanager.h:** Archivo que contiene las cabeceras de las funciones que hará el gestor de archivos.

- **filemanager.cpp:** Implementación del gestor de archivos local. Es decir, gestiona los archivos del servidor. Contiene tres funciones principales y un atributo:
 - **Atributo dirPath:** Contiene el directorio donde están los archivos del servidor
 - **listFiles():** Muestra los nombres de todos los archivos regulares en el directorio del servidor
 - **readFile():** Dado el nombre del archivo, lee y guarda el contenido de dicho archivo
 - **writeFile():** Escribe un archivo con un nombre dado en el directorio del servidor

Cliente

Para el cliente se usan los siguientes archivos:

- **Ficheros utils:** Fichero con funciones de utilidad que nos ayudan sobre todo con asuntos relacionados con la emisión y recepción de datos entre terminales.
- **server.h:** Archivo con la plantilla del servidor, lo necesitamos por las funciones auxiliares y el enumerado fileFuncs.
- **filemanager.h:** Archivo que contiene las cabeceras de las funciones que hará el gestor de archivos.
- **fileManagerDistribuido.cpp:** Implementación distribuida del fileManager. Se conecta con el servidor para hacer las funciones necesarias:
 - **Atributo dirPath:** Contiene el directorio donde están los archivos en el servidor
 - **Constructores:** Inician la conexión con el servidor y inician los atributos necesarios
 - **Destructor:** Al destruir el objeto se cierra la conexión con el servidor
 - **listFiles():** Pide al servidor todos los archivos en su directorio definido en dirPath
 - **readFile():** Dado el nombre del archivo, se le pide al servidor el contenido de dicho artículo.
 - **writeFile():** Escribe un archivo con un nombre dado en el directorio del servidor
- **main_fm.cpp:** Archivo main del cliente con un menú para hacer las siguientes funcionalidades:
 - **ls:** Lista todos los archivos del directorio local
 - **lls:** Lista todos los archivos del directorio del servidor
 - **upload n:** Sube el fichero n al servidor
 - **download n:** Descarga el fichero n del servidor

Implementación Con Broker De Objetos

La siguiente implementación añade complejidad al sistema. Actualmente, podíamos tener varios clientes conectándose a un servidor. Ahora, usamos el bróker para que el cliente pueda elegir a que servidor se quiere conectar. Para ello, implementamos un bróker que almacenara las IPs de los servidores para poder mostrárselas al cliente. Una vez elegido el servidor, se cerrará la conexión con el bróker.

Al tener varias IPs, usaremos AWS para crear todas las maquinas necesarias para servidores y bróker mientras que nuestro cliente se mantendrá en local como nuestro ordenador.

Configuración de AWS

Para montar la arquitectura, necesitamos 3 maquinas EC2 para que actúen como 2 servidores y un bróker. Todas estas maquinas tendrán que pertenecer al mismo security group ya que todas permiten el mismo trafico de entrada y salida. Aparte de esto, cada maquina tiene su propia IP elástica para que la IP de las maquinas no cambie y siempre sea la misma.

☐	Name	ID de la instancia	Estado de la i...	Tipo de inst...	Comprobación de	Estado de la ali	Zona de dispon...	DNS de IPv4 pública	Dirección IP...	IP elástica
☐	Broker-Practica1	i-01385d23b697ec2b0	En ejecución	t3a.medium	3/3 comprobador	Ver alarmas +	us-east-1f	ec2-52-3-251-170.com...	52.3.251.170	52.3.251.170
☐	Server1_Practi...	i-035c2819c3a3ab717	En ejecución	t3a.medium	3/3 comprobador	Ver alarmas +	us-east-1f	ec2-18-233-49-210.co...	18.233.49.210	18.233.49.210
☐	Server2_Practi...	i-0454cb568d8a6cd8c	En ejecución	t3a.medium	3/3 comprobador	Ver alarmas +	us-east-1f	ec2-3-217-142-37.com...	3.217.142.37	3.217.142.37

Configuración Security Group

Para configurar el grupo de seguridad tenemos que tener en cuenta lo siguiente:

- Las conexiones con el bróker se hacen a través del puerto 3000
- Las conexiones con servidores se hacen a través del puerto 3001

Dicho esto tenemos que añadir reglas para que nuestra maquinas permitan trafico de entrada de dichos puertos para que podamos enviar información entre maquinas.

ID de la regla del gr...	Versión de IP	Tipo	Protocolo	Intervalo de puertos	Origen
sgr-0327ad0edca443ae2	IPv4	TCP personalizado	TCP	3001	0.0.0.0/0
sgr-0a2f5c0295d7a2e68	IPv4	SSH	TCP	22	0.0.0.0/0
sgr-0a890c3253f0b43ae	IPv4	TCP personalizado	TCP	3000	0.0.0.0/0

Broker

Tanto cliente como servidor no reciben muchos cambios ya que solo se añade la funcionalidad de conectarse al servidor. Sin embargo, se ha creado un nuevo ejecutable llamado bróker que contenga lo siguiente:

- **Ficheros utils:** Fichero con funciones de utilidad que nos ayudan sobre todo con asuntos relacionados con la emisión y recepción de datos entre terminales.
- **broker.h:** Fichero de cabeceras para la clase broker, además de esto contiene lo siguiente:
 - **Enumerado brokerFuncs:** Contiene todos los mensajes que puede recibir el broker para ejecutar sus funcionalidades más los ack.
 - **getServerToConnect():** Función que ejecutara el cliente para poder conectarse a un servidor
 - **register_server():** Función a la que llama un servidor para poder registrarse en el broker.
 - **close_broker():** Función que permite cerrar la conexión con el broker
- **broker.cpp:** Fichero con la implementación del broker y su main. Contiene lo siguiente:
 - **getNumSalas():** Devuelve el número de salas disponibles
 - **addServer():** Añade un servidor al broker
 - **getServerIP():** Recibe como entrada el numero de sala y devuelve el IP de dicha sala
 - **atenderCliente():** Atiende a un cliente o servidor en función de la petición que envíe.

Conclusión

En el .zip, aparte de este documento existen dos carpetas. Cada carpeta tiene un implementación del código con y sin broker.